Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность АС-68

Выполнил:

Студент 3-го курса

Группы АС-68

Дворак В. А

Проверил:

А. А. Крощенко
Ст. преп. кафедры ИИТ,

«___»_____ 2026 г.

Брест 2026

Цель: На практике сравнить работу нескольких алгоритмов классификации, таких как **метод k-ближайших соседей (k-NN)**, **деревья решений** и **метод опорных векторов (SVM)**. Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 4

Вариант 4

- `Digits`
- Распознать, какая цифра (от 0 до 9) изображена на картинке 8x8 пикселей
- **Задания:**
    1. Загрузите встроенный в `scikit-learn` набор данных `digits`;
    2. Разделите данные на обучающую и тестовую выборки;
    3. Обучите три модели (k-NN, Decision Tree, SVM) для многоклассовой классификации;
    4. Для каждой модели выведите `classification_report` (sklearn.metrics), содержащий основные метрики для каждого класса;
    5. Сравните общую точность моделей и определите, какая из них лучше всего подходит для этой задачи.

Код программы:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score


def main():
    digits = load_digits()

    X = digits.data
    y = digits.target

    print("Набор данных Digits")
    print("Количество объектов:", X.shape[0])
    print("Количество признаков:", X.shape[1])
    print("Классы:", digits.target_names)

    # Заменили 42 на 123
    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.2, random_state=123, stratify=y
    )

    print("\nОбучающая выборка:", X_train.shape[0])
    print("Тестовая выборка:", X_test.shape[0])

    models = {
        "k-NN": make_pipeline(
            StandardScaler(),
            KNeighborsClassifier(n_neighbors=5)
        ),
        "Decision Tree": DecisionTreeClassifier(random_state=123),
        "SVM": make_pipeline(
            StandardScaler(),
            SVC(kernel="rbf")
        )
    }

    results = {}

    for name, model in models.items():
        print("\nМодель:", name)

        model.fit(X_train, y_train)
        predictions = model.predict(X_test)

        accuracy = accuracy_score(y_test, predictions)
        results[name] = accuracy

        print("Общая точность:", round(accuracy, 4))
        print(classification_report(
            y_test, predictions, labels=list(range(10)),
            zero_division=0
        ))

    print("\nИсследование параметра k для k-NN")

    for k in [1, 3, 5, 7, 9, 11]:
        model = make_pipeline(
            StandardScaler(),
            KNeighborsClassifier(n_neighbors=k)
        )

        model.fit(X_train, y_train)
        predictions = model.predict(X_test)
        accuracy = accuracy_score(y_test, predictions)

        print(f"k = {k}: Accuracy = {accuracy:.4f}")

    print("\nСравнение моделей")

    for name, accuracy in sorted(
        results.items(), key=lambda item: item[1], reverse=True
    ):
        print(f"{name}: Accuracy = {accuracy:.4f}")

    best_model = max(results, key=results.get)
    print("Лучшая модель:", best_model)
    print("Её точность:", round(results[best_model], 4))


if __name__ == "__main__":
    main()
```

```
Сравнение моделей
SVM: Accuracy = 0.9861
k-NN: Accuracy = 0.9833
Decision Tree: Accuracy = 0.8778
Лучшая модель: SVM
Её точность: 0.9861
```

Orange:

Python script:

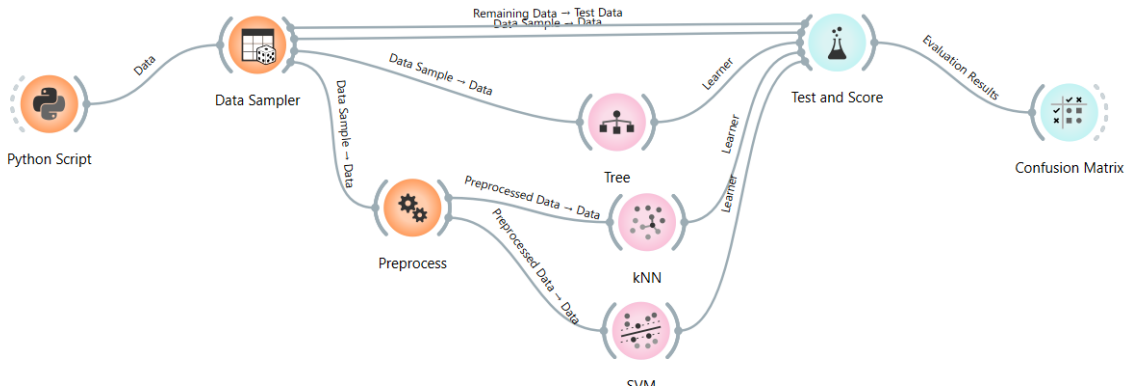
```
import numpy as np
from sklearn.datasets import load_digits
from Orange.data import Domain, Table, ContinuousVariable, DiscreteVariable

digits = load_digits()

attributes = [ContinuousVariable(f"pixel_{i}") for i in range(64)]
class_var = DiscreteVariable("digit", values=[str(i) for i in range(10)])

domain = Domain(attributes, class_var)

out_data = Table.from_numpy(
domain,
digits.data,
digits.target.astype(int)
)
```



Вывод: на практике сравнил работу алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научился подбирать гиперпараметры моделей и оценивать их влияние на результат